

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Completed

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2
3 int main() {
4     int n, i, j;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int at[n], bt[n], ct[n], tat[n], wt[n], completed[n];
10
11     for(i = 0; i < n; i++)
12         completed[i] = 0;
13
14     for(i = 0; i < n; i++) {
15         printf("Enter Arrival Time and Burst Time for process %d: ", i+1);
16         scanf("%d %d", &at[i], &bt[i]);
17     }
18
19     int time = 0, done = 0;
20     float total_tat = 0, total_wt = 0;
21
22     while(done < n) {

```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Completed

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

Select Language : C (GCC 9.2.0)

```

32     }
33
34     if(index == -1) {
35         time++;
36     } else {
37         time += bt[index];
38         ct[index] = time;
39
40         tat[index] = ct[index] - at[index];
41         wt[index] = tat[index] - bt[index];
42
43         total_tat += tat[index];
44         total_wt += wt[index];
45
46         completed[index] = 1;
47         done++;
48     }
49 }
50
51 printf("\nAverage Turnaround Time = %.2f\n", total_tat/done);
52 printf("Average Waiting Time = %.2f\n", total_wt/done);
53
54 return 0;
55 }

```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success ✓

Input :

Expected Output :

4
0 6
0 8
0 7
0 3

Enter number of
processes:
Enter Arrival Time and
Burst Time for P1:
Enter Arrival Time and
Burst Time for P2:
Enter Arrival Time and
Burst Time for P3: